

轮趣科技

WHEELTEC Self-Balancing Bike Circuit and Programming Handbook

Recommended to follow our official account to get updated information



Version Description:

Version	date	Contents
V1.0	2022/06/23	First release

Website: www.wheeltec.net

Preface

This document mainly explains the circuit and software development of the WHEELTEC Z100 servo self-balancing bicycle.

Design ideas and implementation, circuit design including the description and principle of each electronic module, software development including STM32

The design distribution of microcontroller resources and the implementation principles of each part of the program.

Finally, a flow chart of bicycle balance control is provided, and the content of the LQR balance control principle section is provided.

In another document [3. Self-balancing unmanned bicycle based on handlebar steering control.docx].

Preface.....	2
1. Circuit Design.....	4
1.1 MCU Minimum System Design.....	4
1.2 Gyroscope Circuit Design.....	4
1.3 Motor drive circuit design.....	5
1.4 Motor Encoder.....	6
1.5 Potentiometer Circuit Design.....	7
1.6 Servo Circuit Design.....	8
1.7 Power Circuit Design.....	8
2. Software Development.....	9
2.1 Design of the car owner program.....	9
2.2 Design of car control interrupt task program.....	10
2.3 Design of motor and servo driver.....	10
2.4 Motor encoder reading program design.....	11
2.5 Bluetooth Programming.....	12
2.6 ADC acquisition battery voltage and potentiometer program design.....	13
3. Car control flow chart.....	14

1. Circuit design

1.1 MCU Minimum System Design

The servo car uses STM32F103C8T6 as the main control, and its schematic diagram is shown in Figure 1-1.

The input voltage is 5V or 3.3V. The power supply uses the AMS1117 chip to convert 5V to 3.3V.

For STM32 chips and external modules. In addition, the main control board is equipped with a one-key download serial port circuit, a

User buttons and a reset button, a programmable LED, and a power indicator.

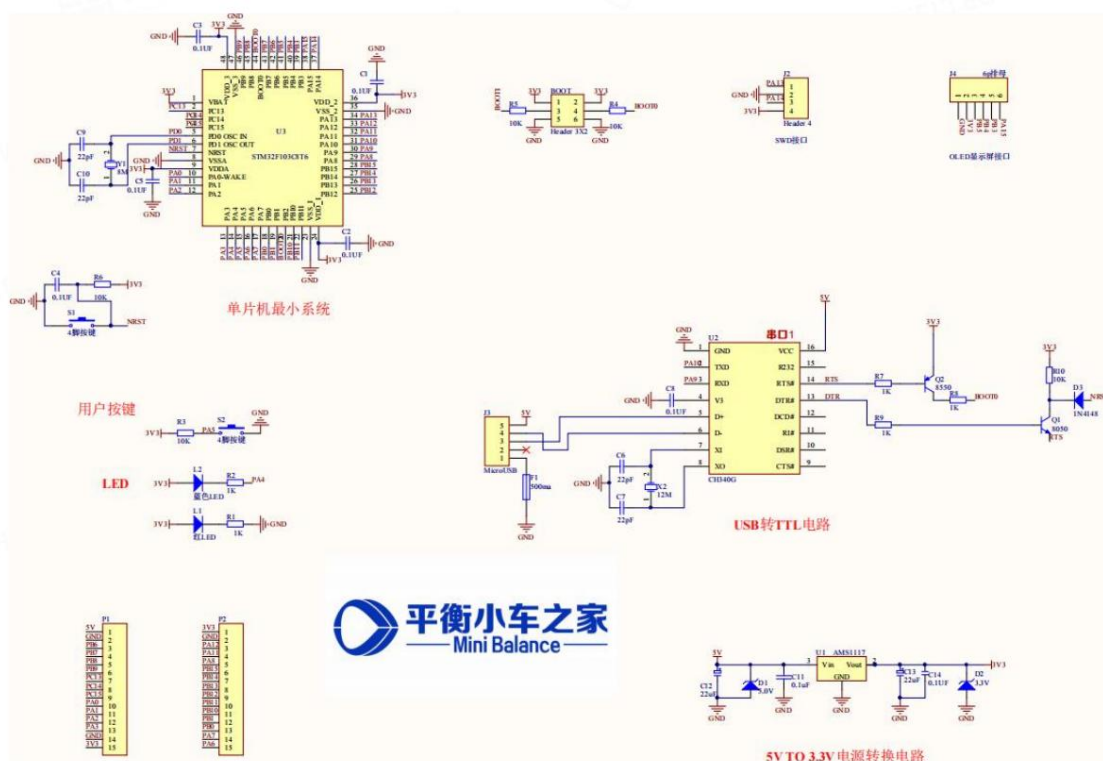


Figure 1-1 STM32 minimum system design

1.2 Gyroscope Circuit Design

The car uses the MPU6050 chip as a gyroscope sensor, which has 3-axis angular velocity and 3-axis acceleration.

Speed sensor, using IIC protocol communication, with built-in digital motion processor (DMP) hardware acceleration

Engine, can directly output 6-axis attitude solution data; in order to meet the needs and measurement accuracy, the sensor

Provides different ranges, gyroscope range from $\pm 250^\circ$ to $\pm 2000^\circ$, accelerometer range from $\pm 2g$

The AD0 pin can be changed to $\pm 16g$, which meets most usage scenarios.

Change the address of the chip in IIC communication. When AD0 is connected to GND, the address is 0x68, and when it is connected to VCC, the address is 0x69.

The INT pin is the interrupt pin of the chip. When the chip data is ready, the pin will change from high level to low level.

The data ready frequency is the same as the sampling frequency configured by the chip; SDA and SCL are IIC communication pins.

It is connected to the common IO of the microcontroller, so when writing the program, it will communicate with the chip by simulating IIC.

Line communication.

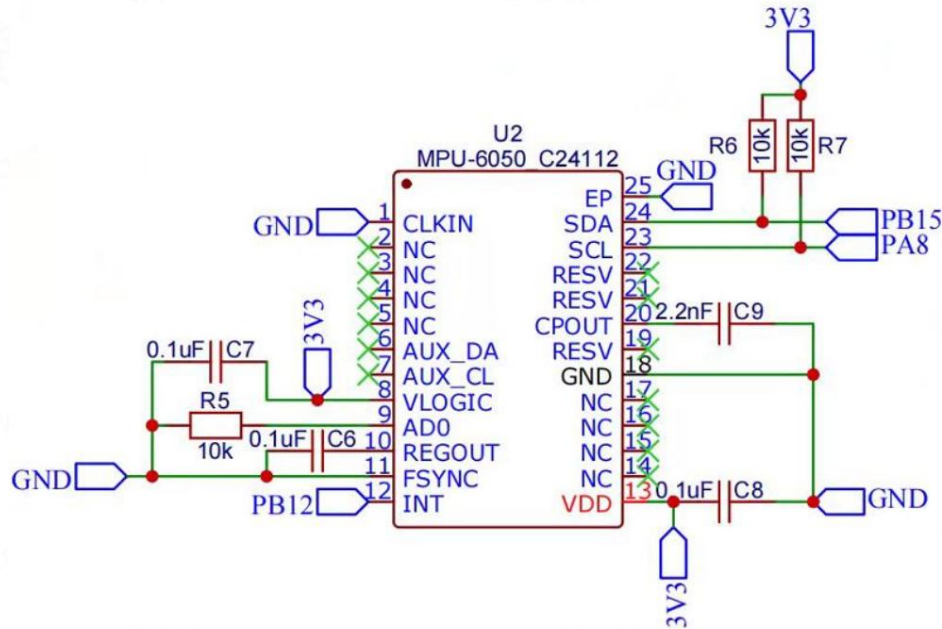


Figure 1-2 MPU6050 connection diagram

1.3 Motor drive circuit design

The standard version of the servo car only uses one motor for rear wheel drive.

AT8236 has an H-bridge structure composed of 4 MOS tubes, as shown in Figure 1-3.

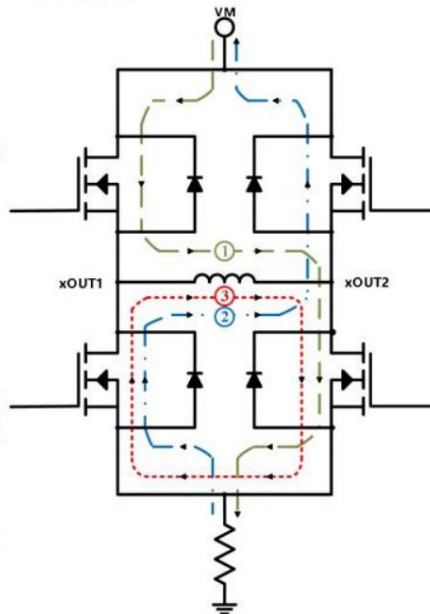


Figure 1-3 Schematic diagram of the internal H-bridge of AT8236

This chip can use ordinary TTL level to drive the motor (the motor speed cannot be adjusted), and can also be used

PWM input is used to drive the motor. PWM drive is mainly used here. The schematic diagram is shown in Figure 1-4.

The PWM control truth table is as follows:

Table 1-1 AT8236 truth table

IN1	IN2	Function
PWM	0	Forward PWM, fast decay
1	PWM	Forward PWM, slow decay
0	PWM	Reverse PWM, fast decay
PWM	1	Reverse PWM, slow decay

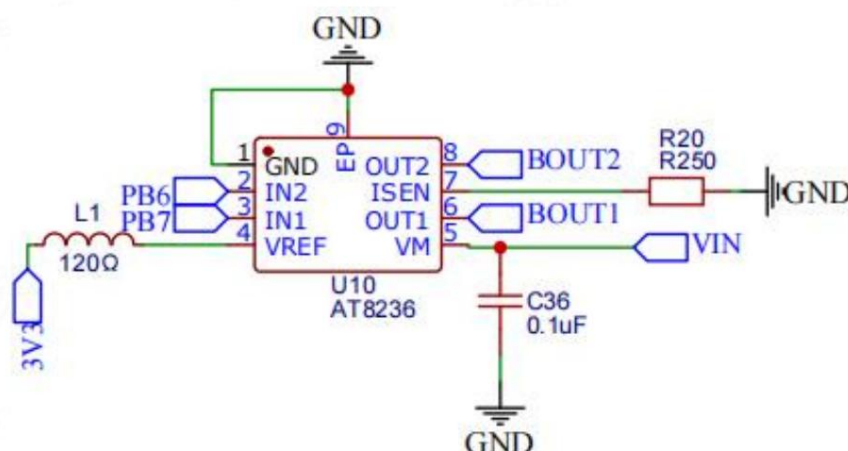


Figure 1-4 AT8236 schematic diagram

1.4 Motor encoder

The motor used in the car is integrated with a GMR encoder. The rotation of the motor can make the encoder output pulses.

The pulse is divided into two phases, AB, and there is a 90° phase difference between the two pulses. The schematic diagram is shown in Figure 1-5.

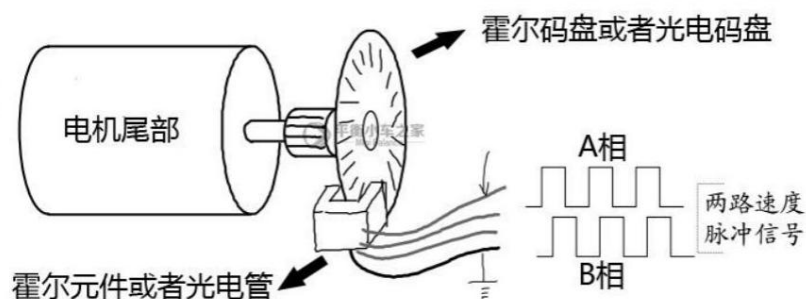


Figure 1-5 The motor rotates so that the encoder outputs pulses

We can obtain the motor speed by reading the pulse count and perform closed-loop control on the motor.

The relationship between the number of pulses and the motor reduction ratio is as follows:

The number of pulses output by the encoder when the motor rotates one circle = motor reduction ratio $\times$ encoder accuracy (number of lines)

In the program, we use the STM32 timer to read the number of pulses output by the encoder. The advantage is that it does not require

It takes up CPU resources, is read by hardware, and can be configured as 4 times the frequency, which improves the accuracy of the encoder.

So we need to lead out the two pins of the timer and provide power to the encoder. The schematic diagram is as follows

As shown in Figure 1-6.

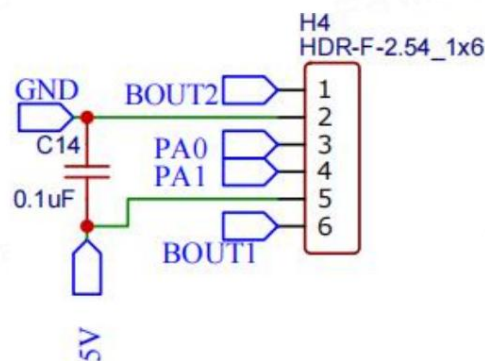


Figure 1-6 Motor encoder and positive and negative pole interface

1.5 Potentiometer Circuit Design

The servo car contains two potentiometers, one for adjusting the initial position of the servo, and one for adjusting the

The purpose of adjusting these two potentiometers is to make the car balanced without

The program is re-modified to facilitate user use. Its schematic diagram is shown in Figure 1-7. The maximum potentiometer used in the figure

The resistance value is 10k, and a 10k resistor is connected in series, so the voltage range on the potentiometer is 0~1.65V.

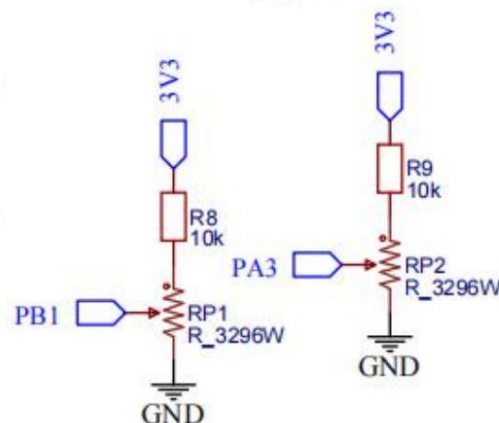


Figure 1-7 Potentiometer schematic diagram

1.6 Servo Circuit Design

The servo drive only needs one PWM to complete the control, where the PWM duty cycle is related to the servo rotation angle.

The relationship is shown in Figure 1-8. It should be noted that the PWM frequency of the servo is generally 50Hz.

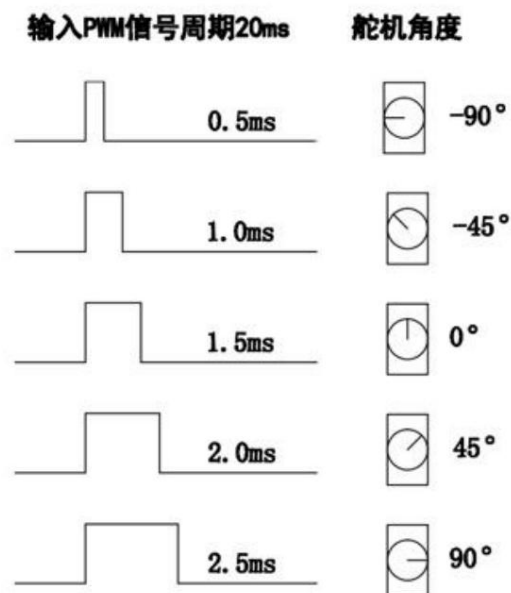


Figure 1-8 Relationship between PWM duty cycle and angle of input servo

Therefore, the control of the servo only requires the power line and a timer pin. The schematic diagram is shown in Figure 1-9

shown.

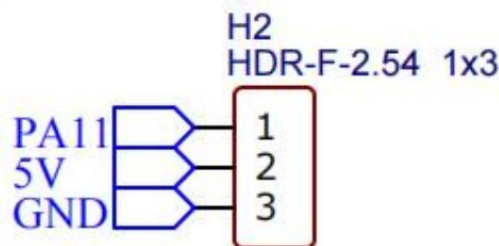


Figure 1-9 Schematic diagram of the servo interface

1.7 Power Circuit Design

The car power supply uses RT8289GSP step-down chip for voltage stabilization. This chip supports a wide range of 5~32V.

Voltage input, after the chip voltage is stabilized, it can output 5V5A power externally; in addition, the power input terminal is connected in series

A 10k and a 1k resistor are used to divide the battery voltage, making it easier for the microcontroller to perform sampling calculations.

Its schematic diagram is shown in Figure 1-10. It should be noted that the power supply voltage will be input to the motor driver chip and then

Add to the motor. The voltage applied to the motor of the car should not be too high. The recommended voltage range is 7~12V.

It is powered by a 2s aircraft model battery, and the voltage at full charge is 8.4V.

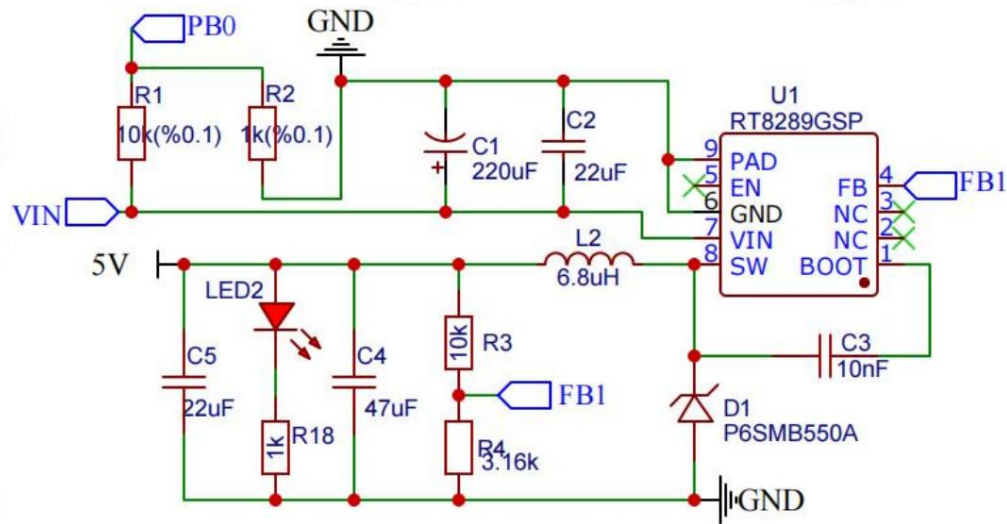


Figure 1-10 Power supply circuit schematic

2. Software Development

2.1 Car Owner Program Design

The initialization of all peripherals of the car is performed in the main program. The following is the initialization of the microcontroller resources used.

illustrate:

USART1: used to send data to the host computer and display waveform

USART3: used for communication between Bluetooth app and car

TIM4: Use TIM4CH1 and TIM4CH2 to generate PWM to drive the rear wheel motor

TIM2: Initialized to quadrature encoder mode, used to collect the number of pulses output by the motor encoder

ADC: Use channels 3, 8, and 9 of ADC1 to collect the potentiometer,

Battery voltage, potentiometer to modify the initial position of the servo

TIM1: Use TIM1CH4 to generate PWM to drive the servo

EXTI: configure PB12 to map to external interrupt line 12, triggered by the INT pin of MPU6050 for 5ms

External interrupt

IIC: The car does not use hardware IIC, but uses PA8 and PB15 to simulate IIC communication

SPI: The car does not use hardware SPI, but is simulated by PB3, 4, 5 and PA15 for OLED

show

SWD: Disable JTAG function and enable SWD debugging function, which allows online debugging and program downloading.

After initializing the above MCU resources, it starts to enter the infinite loop function.

The display mode of the car is determined by the bit, which is divided into two types: app data sending + oled refresh and host computer display waveform

The mode can be selected, and the mode switching is changed by the button. In addition, when Bluetooth sends a power-down save command, the dead loop

The data write flag in the ring will be set to 1, and the pid parameters of the car will be written to Flash to achieve power failure

Save function.

2.2 Design of car control interrupt task program

The control interrupt task of the car is provided by the external interrupt triggered by MPU6050. The principle is:

After setting the STM32 external interrupt, set the acquisition frequency of MPU6050 and turn on MPU6050

Interrupt function, at this time, the INT pin of MPU6050 will trigger the external

Interrupt.

The configuration of external interrupt is completed in MiniBalance_EXTI_Init() of the main function. The interrupt frequency

It can be modified by the following macro definition. The location is in the MPU6050.c file.

```
#define DEFAULT_MPU_HZ (200)
```

In the 5ms external interrupt task, the MCU will collect the battery voltage and the data of the two potentiometers.

data, obtain the car's posture information, read the encoder data, and perform PID+LQR through these data

Calculate and finally output the calculation results to the motor and servo.

2.3 Design of motor and servo driver

In Section 1.3, we know the motor truth table and motor drive schematic. We need to convert TIM4CH1,

TIM4CH2 is configured in PWM output mode to control the forward and reverse rotation and speed of the motor.

In the MiniBalance_PWM_Init(7199,0) function of the main function, the specific configuration process is not expanded here.

The configuration result is that channels 1 and 2 of TIM4 are configured as PWM1 mode, and the PWM frequency is 10KHz.

0~7200 corresponds to PWM duty cycle 0~100%.

In Section 1.6, we know the control method of the servo and the control interface of the servo is PA11, so we need to

TIM1CH4 is configured as PWM output mode by Servo_PWM_Init(9999,144) in the main function

Function configuration, the configuration result is that PA11 outputs a PWM signal with a frequency of about 50Hz, 0~10000 corresponds to

0~100% duty cycle.

After the timer is initialized, the motor forward and reverse control is programmed according to the motor drive truth table.

The following is the implementation process:

PWMA1 corresponds to PB7, PWMA2 corresponds to PB6, SERVO corresponds to PA11, these three macros

What is defined and modified are the corresponding PWM duty cycles to achieve motor speed control and servo angle control.

```
void Set_Pwm(int moto1) {

    if(moto1>0)          PWMA1=7200,          PWMA2=7200-moto1;
    else                 PWMA2=7200,          PWMA1=7200+moto1;

    SERVO=Servo + SERVO_BIAS; //Servo control
}
```

2.4 Motor encoder reading program design

In [Section 1.4](#), we know the characteristics of the encoder. We need a timer to calculate the encoder AB terms.

From the encoder interface schematic diagram, we know that we need to configure TIM2 to quadrature encoder mode.

It should be noted that only channel 1 and channel 2 of the timer have encoder counting function.

The Encoder_Init_TIM2() function is implemented, and the configuration result is to configure the TIM2 timer as the encoder mode.

Formula 3, that is, the encoder can count at each pulse edge, thereby achieving 4 times the frequency and improving the encoder

Figure 2-1 below shows how the microcontroller's timer reads the encoder pulses.

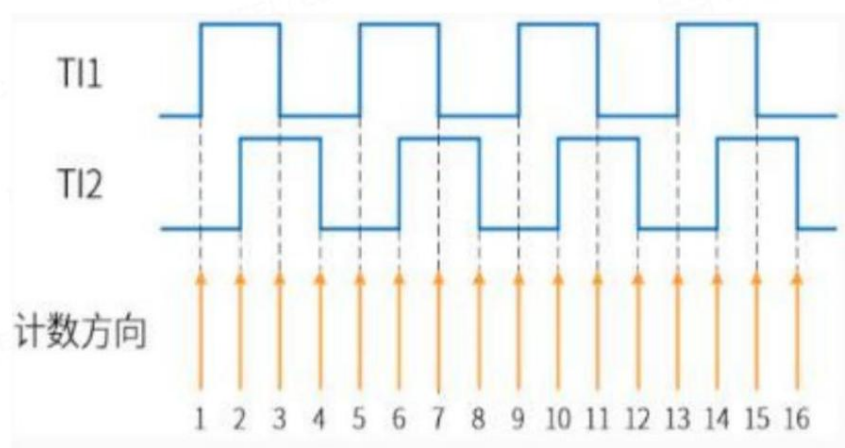


Figure 2-1 Schematic diagram of timer reading encoder pulse

Note: When the motor's rotation direction changes, the phase difference between TI1 and TI2 will also change, and the counting direction will change.

The direction will also change.

After initialization is completed, the encoder pulse counting is completed by the timer hardware, and we do not need to write

For any program, we only need to read the value of the counter in the timer. Here is the implementation process:

```
int Read_Encoder(u8 TIMX) {

    int Encoder_TIM;
    switch(TIMX)
```

```

    {
        case 2: Encoder_TIM= (short)TIM2 -> CNT; TIM2 -> CNT=0;break; case 3: Encoder_TIM=
            (short)TIM3 -> CNT; TIM3 -> CNT=0;break; case 4: Encoder_TIM= ( short)TIM4 -> CNT;
            TIM4 -> CNT=0;break; default: Encoder_TIM=0;

    }

    return Encoder_TIM;
}

```

2.5 Bluetooth Programming

The Bluetooth module is used for communication between the APP and the car, and can control the movement, steering and viewing of the car.

The Bluetooth module communicates with the mobile phone via Bluetooth.

The microcontroller uses serial port 3 to communicate, which is like a bridge between a mobile phone and the microcontroller. Initialization is done by the main function

The number of `uart3_init(9600)` is completed.

The microcontroller receives the data from the Bluetooth APP in the interrupt service function of serial port 3, which is located in `uart3.c`

The void `USART3_IRQHandler(void)` function in the file is not expanded here. The implementation of the microcontroller sending data to the Bluetooth

APP is in the `APP_Show()` function in `main.c`. It is not expanded here.

The detailed usage instructions are in the help instructions.

The following explains how to control the car's steering after receiving Bluetooth APP data. The function is implemented as follows:

```

void Get_RC(void) {

    static u8 TurnFlag=0;
    static float turn; //Receive
    the app control signal and read the mechanical median
    value once if((Flag_Right||Flag_Left)&&TurnFlag==0) {

        turn = balance_point;
        TurnFlag = 1;

    } //Left and
    right control          balance_point = turn + LeftControl;
    if(Flag_Left) else if (Flag_Right) balance_point = turn - RightControl;
    //APP has no control, mechanical
    midpoint reset if(Flag_Right==0&&Flag_Left==0&&TurnFlag==1)
    {
        balance_point = turn;
        TurnFlag = 0;
    }
}

```

Controlling the car's steering is achieved by modifying the car's mechanical median. When the app receives the steering data

After the data is received, the program will first save the current mechanical median value of the car, and then determine which direction to turn to.

The mechanical median plus the deviation in the corresponding direction can complete the steering; after the app is out of control, the median is restored

Return to the original value to stop steering.

2.6 ADC acquisition battery voltage and potentiometer program design

The analog quantity acquisition of the car is completed using ADC1, and the ADC is initialized by the main function

The configuration process is not expanded here. The configuration result is that the ADC data is right-aligned.

Non-scanning mode, non-continuous conversion mode, acquisition is triggered by software. To complete multi-channel acquisition without

When using the scanning mode, an interface needs to be provided to the user, and the corresponding channel is opened when the channel to be collected is input.

Channel scanning, the function implementation process is as follows:

```
u16 Get_Adc(u8 ch) {

    //Set the specified ADC's rule group channel, a sequence, and
    sampling timeADC-RegularChannelConfig(ADC1, ch, 1, ADC_SampleTime_239Cycles5);
    ADC_SoftwareStartConvCmd(ADC1, ENABLE); //Enable the specified ADC1's software conversion start
    functionwhile(!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC)); //Wait for the conversion
    to endreturn ADC_GetConversionValue(ADC1); //Return the most recent conversion result of the ADC1 rule group
}
```

The implementation principle of this function is: in the non-scanning mode of ADC, pass in the channel that the user needs to collect,

The channel will be set as acquisition sequence 1 in the function, and then the software triggers the conversion and returns the result.

The data of the two potentiometers and the battery voltage can be collected by simply transferring them to the corresponding channels.

Relationship between battery voltage acquisition result and actual battery voltage:

As can be seen from Section 1.7, the input voltage is divided by a series connection of 10k and 1k resistors and then connected to the microcontroller.

From a simple circuit analysis, we can see that the currents in the series circuits are equal, and the internal voltage relationships are as follows:

Lower: $U_{\text{Actual voltage}} = 11 \cdot U_{\text{ADC}}$. So multiply the collected voltage by 11 times to get the battery voltage.

Since the ADC data register is 12 bits, the highest voltage it can measure is 3.3V, so the final voltage is

The cell voltage is calculated as:

$$U_{\text{actual voltage}} = \text{Get_Adc}(\text{Battery_Ch}) * 3.3 * 11 / 4096$$

The principles of the other two potentiometers are the same as those of battery voltage acquisition, and the mechanical median is completed through a linear relationship.

The adjustment of the initial position of the servo is implemented in the program as follows:

Servo adjustment: $\text{SERVO_BIAS} = (\text{Get_Adc}(\text{Adjust_Ch}) - 1024) / 12;$

Mechanical median adjustment: $\text{BalancePoint_Offset} = (5.06f / 1024.0f) * (\text{Get_Adc}(\text{ZhongZhi_Ch}) - 1024);$

3. Car control flow chart

The car program flow chart can be divided into two parts. The first is the while(1) infinite loop of the main program.

One is the interrupt processing data and control part. The main program flow chart is shown in Figure 3-1. After the system is powered on,

All peripherals on the car will be initialized, including delay function configuration and initialization of corresponding timer implementations.

The corresponding functions are: ADC initialization, analog SPI initialization, analog IIC initialization, MPU6050 initialization

After the initialization is completed, it will enter the while(1) infinite loop, in which it will pass

The flag bit is used to determine whether to send app data and refresh the OLED, or to send data to the host computer to display the waveform.

(This flag is controlled by a button).

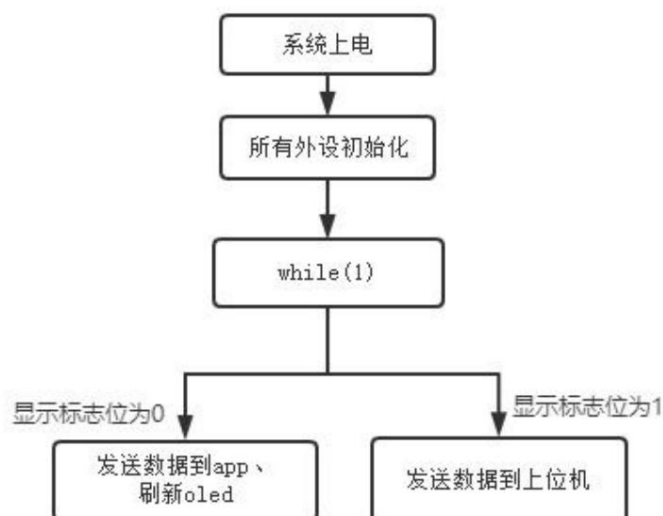


Figure 3-1 Main program flow chart

The control part of the car is completed in the interrupt, which is provided by the INT interrupt pin of MPU6050.

The interrupt frequency can be changed by configuring the sampling frequency of MPU6050. In this program, the interrupt frequency is

200Hz (1 interrupt every 5ms). Its control flow chart is shown in Figure 3-2. The control part includes all sensors

Data reading, PID+LQR calculation, motor and servo control, receiving Bluetooth app data, etc.

Allow.

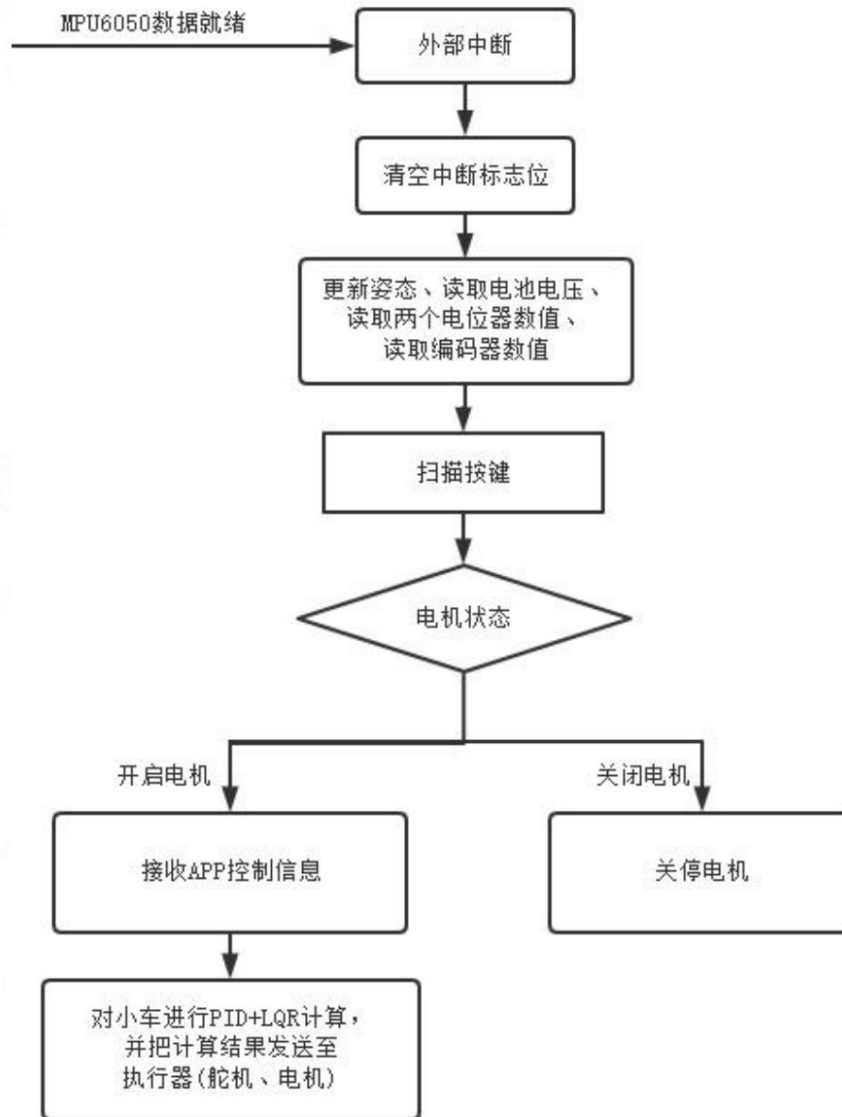


Figure 3-2 Interrupt task program flow chart